Namespace CloudyWing.DatabaseFacade

Classes

CommandExecutor

Provides a fluent facade for executing ADO.NET commands with shared configuration and parameter handling.

FacadeConfiguration

Provides global defaults used when creating CommandExecutor instances.

KeepConnectionRequiredException

The exception that is thrown when an operation requires KeepConnection to be enabled.

ParameterCollection

Represents the parameters queued for the next CommandExecutor operation.

ParameterMetadata

Describes the values used to create and configure a provider-specific database parameter.

Enums

ResetItems

Specifies which parts of a CommandExecutor should be reset after an operation completes.

Class CommandExecutor

Namespace: CloudyWing.DatabaseFacade

Assembly: CloudyWing.DatabaseFacade.dll

Provides a fluent facade for executing ADO.NET commands with shared configuration and parameter handling.

```
public sealed class CommandExecutor : IDisposable
```

Inheritance

object ← CommandExecutor

Implements

IDisposable

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

CommandExecutor(DbProviderFactory?, string?, bool?)

Initializes a new instance of the CommandExecutor class.

```
public CommandExecutor(DbProviderFactory? providerFactory, string? connStr, bool?
keepConnection = null)
```

Parameters

`providerFactory` DbProviderFactory

The provider factory used to create connections and commands. When null, the configured default factory is used.

`connStr` string

The connection string to use. When null, the configured default connection string is used.

`keepConnection` bool?

Overrides the configured keep-connection behavior when a value is supplied.

CommandExecutor(bool?)

Initializes a new instance of the CommandExecutor class by using the configured defaults.

```
public CommandExecutor(bool? keepConnection = null)
```

Parameters

`keepConnection` bool?

Overrides the configured keep-connection behavior when a value is supplied.

CommandExecutor(string?, bool?)

Initializes a new instance of the CommandExecutor class by using the configured provider factory.

```
public CommandExecutor(string? connStr, bool? keepConnection = null)
```

Parameters

`connStr` string

The connection string to use. When null, the configured default connection string is used.

`keepConnection` bool?

Overrides the configured keep-connection behavior when a value is supplied.

Properties

CommandText

Gets or sets the SQL statement, stored procedure name, or other provider-specific command text to execute.

```
public string? CommandText { get; set; }
```

Property Value

string

CommandTimeout

Gets or sets the command timeout, in seconds.

```
public int CommandTimeout { get; set; }
```

Property Value

int

CommandType

Gets or sets the command type used when executing CommandText.

```
public CommandType CommandType { get; set; }
```

Property Value

CommandType

Connection

Gets the current connection instance when one has been created.

```
public IDbConnection? Connection { get; }
```

Property Value

IDbConnection

ConnectionString

Gets the connection string used when opening the underlying connection.

```
public string? ConnectionString { get; }
```

Property Value

string

DbProviderFactory

Gets the database provider factory used to create connections and commands.

```
public DbProviderFactory? DbProviderFactory { get; }
```

Property Value

DbProviderFactory

KeepConnection

Gets a value indicating whether the underlying connection should remain open after command execution.

```
public bool KeepConnection { get; }
```

Property Value

bool

Parameters

Gets the parameters that will be applied to the next command execution.

```
public ParameterCollection Parameters { get; }
```

Property Value

ParameterCollection

Transaction

Gets the active transaction, or null when no usable transaction exists.

```
public IDbTransaction? Transaction { get; }
```

Property Value

IDbTransaction

Methods

BeginTransaction(IsolationLevel?)

Begins a transaction on the current connection.

```
public IDbTransaction BeginTransaction(IsolationLevel? level = null)
```

Parameters

`level` IsolationLevel?

The isolation level to use. When null, DefaultIsolationLevel is used.

Returns

IDbTransaction

The newly created transaction.

Exceptions

KeepConnectionRequiredException

Thrown when KeepConnection is false.

CreateDataReader(ResetItems, CommandBehavior)

Executes the current command and returns a data reader.

```
public IDataReader CreateDataReader(ResetItems thenReset = ResetItems.All, CommandBehavior
behavior = CommandBehavior.SequentialAccess)
```

Parameters

`thenReset` ResetItems

Specifies which command state values are reset after the command succeeds.

`behavior` CommandBehavior

The CommandBehavior flags applied to the reader.

Returns

IDataReader

An IDataReader for the executed command. The caller is responsible for disposing the returned reader.

CreateDataReaderAsync(ResetItems, CommandBehavior, CancellationToken)

Executes the current command asynchronously and returns a data reader.

```
public Task<DbDataReader> CreateDataReaderAsync(ResetItems thenReset = ResetItems.All,
CommandBehavior behavior = CommandBehavior.SequentialAccess, CancellationToken
cancellationToken = default)
```

Parameters

`thenReset` ResetItems

Specifies which command state values are reset after the command succeeds.

`behavior` CommandBehavior

The CommandBehavior flags applied to the reader.

`cancellationToken` CancellationToken

The token used to cancel the asynchronous operation.

Returns

Task<DbDataReader>

A task that returns a DbDataReader for the executed command. The caller is responsible for disposing the returned reader.

CreateDataTable(ResetItems)

Executes the current command and materializes the first result set into a DataTable.

```
public DataTable CreateDataTable(ResetItems thenReset = ResetItems.All)
```

Parameters

`thenReset` ResetItems

Specifies which command state values are reset after the command succeeds.

Returns

DataTable

A populated DataTable.

CreateDataTableAsync(ResetItems, CancellationToken)

Executes the current command asynchronously and materializes the first result set into a DataTable.

```
public Task<DataTable> CreateDataTableAsync(ResetItems thenReset = ResetItems.All,
CancellationToken cancellationToken = default)
```

Parameters

`thenReset` ResetItems

Specifies which command state values are reset after the command succeeds.

`cancellationToken` CancellationToken

The token used to cancel the asynchronous operation.

Returns

Task<DataTable>

A task that returns a populated DataTable.

Dispose()

Releases the underlying connection and transaction resources.

```
public void Dispose()
```

Execute(ResetItems)

Executes the current command and returns the number of affected rows.

```
public int Execute(ResetItems thenReset = ResetItems.All)
```

Parameters

`thenReset` ResetItems

Specifies which command state values are reset after the command succeeds.

Returns

int

The number of rows affected.

ExecuteAsync(ResetItems, CancellationToken)

Executes the current command asynchronously and returns the number of affected rows.

```
public Task<int> ExecuteAsync(ResetItems thenReset = ResetItems.All, CancellationToken
cancellationToken = default)
```

Parameters

`thenReset` ResetItems

Specifies which command state values are reset after the command succeeds.

`cancellationToken` CancellationToken

The token used to cancel the asynchronous operation.

Returns

Task <int>

A task that returns the number of rows affected.

~CommandExecutor()

Finalizes an instance of the CommandExecutor class.

```
protected ~CommandExecutor()
```

Initialize(ResetItems)

Resets the selected command state values to their configured defaults.

```
public void Initialize(ResetItems items = ResetItems.All)
```

Parameters

`items` ResetItems

The command state values to reset.

QueryScalar(ResetItems)

Executes the current command and returns the first column of the first row in the result set.

```
public object? QueryScalar(ResetItems thenReset = ResetItems.All)
```

Parameters

`thenReset` ResetItems

Specifies which command state values are reset after the command succeeds.

Returns

object

The first column of the first row in the result set.

QueryScalarAsync(ResetItems, CancellationToken)

Executes the current command asynchronously and returns the first column of the first row in the result set.

```
public Task<object?> QueryScalarAsync(ResetItems thenReset = ResetItems.All,
CancellationToken cancellationToken = default)
```

Parameters

`thenReset` ResetItems

Specifies which command state values are reset after the command succeeds.

`cancellationToken` CancellationToken

The token used to cancel the asynchronous operation.

Returns

Task<object>

A task that returns the first column of the first row in the result set.

SetCommandText(string, CommandType?)

Sets the command text and optionally updates the command type.

```
public CommandExecutor SetCommandText(string commadText, CommandType? commandType = null)
```

Parameters

`commadText` string

The command text to execute.

`commandType` CommandType?

The command type to apply. When null, the existing CommandType value is preserved.

Returns

CommandExecutor

The current CommandExecutor instance.

SetCommandTimeout(int)

Sets the command timeout.

```
public CommandExecutor SetCommandTimeout(int second)
```

Parameters

`second` int

The timeout, in seconds.

Returns

CommandExecutor

The current CommandExecutor instance.

Class FacadeConfiguration

Namespace: CloudyWing.DatabaseFacade

Assembly: CloudyWing.DatabaseFacade.dll

Provides global defaults used when creating CommandExecutor instances.

```
public static class FacadeConfiguration
```

Inheritance

object ← FacadeConfiguration

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

DefaultCommandTimeout

Gets or sets the default command timeout, in seconds.

```
public static int DefaultCommandTimeout { get; set; }
```

Property Value

int

DefaultConnectionString

Gets or sets the default connection string.

```
public static string? DefaultConnectionString { get; set; }
```

Property Value

string

DefaultDbProviderFactory

Gets or sets the default database provider factory used to create connections and commands.

```
public static DbProviderFactory? DefaultDbProviderFactory { get; set; }
```

Property Value

DbProviderFactory

DefaultIsolationLevel

Gets or sets the default isolation level used by BeginTransaction(IsolationLevel?).

```
public static IsolationLevel DefaultIsolationLevel { get; set; }
```

Property Value

IsolationLevel

DefaultKeepConnection

Gets or sets a value indicating whether connections stay open by default after command execution.

```
public static bool DefaultKeepConnection { get; set; }
```

Property Value

bool

OnCommandCreated

Gets or sets the callback invoked after a command has been created and populated.

```
public static Action<IDbCommand>? OnCommandCreated { get; set; }
```

Property Value

Action <IDbCommand>

OnCommandCreating

Gets or sets the callback invoked before a command is created, so the queued parameters and SQL text can be inspected.

```
public static Action<ParameterCollection, string?>? OnCommandCreating { get; set; }
```

Property Value

Action <ParameterCollection, string>

ParameterNamePrefix

Gets or sets the prefix used when an IEnumerable parameter is expanded into multiple provider parameters. Expanded names follow the pattern `{ParameterNamePrefix}{parameterName}{index}`.

```
public static string ParameterNamePrefix { get; set; }
```

Property Value

string

Methods

SetConfiguration(DbProviderFactory, string)

Sets the minimum configuration required to start using CommandExecutor.

```
public static void SetConfiguration(DbProviderFactory dbProviderFactory,
```

```
  string connectionString)
```

Parameters

`dbProviderFactory` DbProviderFactory

The provider factory used to create ADO.NET objects.

`connectionString` string

The default connection string.

Class KeepConnectionRequiredException

Namespace: CloudyWing.DatabaseFacade

Assembly: CloudyWing.DatabaseFacade.dll

The exception that is thrown when an operation requires KeepConnection to be enabled.

```
[Serializable]
public class KeepConnectionRequiredException : Exception, ISerializable
```

Inheritance

object ← Exception ← KeepConnectionRequiredException

Implements

ISerializable

Inherited Members

Exception.GetBaseException() , Exception.GetObjectData(SerializationInfo, StreamingContext) , Exception.GetType() , Exception.ToString() , Exception.Data , Exception.HelpLink , Exception.HResult , Exception.InnerException , Exception.Message , Exception.Source , Exception.StackTrace , Exception.TargetSite , Exception.SerializeObjectState , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.MemberwiseClone() , object.ReferenceEquals(object, object)

Constructors

KeepConnectionRequiredException()

Initializes a new instance of the KeepConnectionRequiredException class.

```
public KeepConnectionRequiredException()
```

KeepConnectionRequiredException(SerializationInfo, StreamingContext)

Initializes a new instance of the KeepConnectionRequiredException class with serialized data.

```
protected KeepConnectionRequiredException(SerializationInfo info, StreamingContext context)
```

Parameters

`info` SerializationInfo

The object that holds the serialized exception data.

`context` StreamingContext

The contextual information about the source or destination.

KeepConnectionRequiredException(string)

Initializes a new instance of the KeepConnectionRequiredException class with a specified error message.

```
public KeepConnectionRequiredException(string message)
```

Parameters

`message` string

The message that describes the error.

KeepConnectionRequiredException(string, KeepConnectionRequiredException)

Initializes a new instance of the KeepConnectionRequiredException class with a specified error message and inner exception.

```
public KeepConnectionRequiredException(string message, KeepConnectionRequiredException
inner)
```

Parameters

`message` string

The message that describes the error.

`inner` KeepConnectionRequiredException

The exception that caused the current exception.

See Also

Exception

Class ParameterCollection

Namespace: CloudyWing.DatabaseFacade

Assembly: CloudyWing.DatabaseFacade.dll

Represents the parameters queued for the next CommandExecutor operation.

```
public sealed class ParameterCollection : KeyedCollection<string, ParameterMetadata>,
IList<ParameterMetadata>, ICollection<ParameterMetadata>, IReadOnlyList<ParameterMetadata>,
IReadOnlyCollection<ParameterMetadata>, IEnumerable<ParameterMetadata>, IList,
ICollection, IEnumerable
```

Inheritance

object ← Collection<ParameterMetadata> ← KeyedCollection<string, ParameterMetadata> ← ParameterCollection

Implements

IList<ParameterMetadata>, ICollection<ParameterMetadata>, IReadOnlyList<ParameterMetadata>, IReadOnlyCollection<ParameterMetadata>, IEnumerable<ParameterMetadata>, IList, ICollection, IEnumerable

Inherited Members

KeyedCollection<string, ParameterMetadata>.Contains(string) , KeyedCollection<string, ParameterMetadata>.Remove(string) , KeyedCollection<string, ParameterMetadata>.Comparer , KeyedCollection<string, ParameterMetadata>.this[string] , Collection<ParameterMetadata>.Clear() , Collection<ParameterMetadata>.Contains(ParameterMetadata) , Collection<ParameterMetadata>.CopyTo(ParameterMetadata[], int) , Collection<ParameterMetadata>.GetEnumerator() , Collection<ParameterMetadata>.IndexOf(ParameterMetadata) , Collection<ParameterMetadata>.Insert(int, ParameterMetadata) , Collection<ParameterMetadata>.Remove(ParameterMetadata) , Collection<ParameterMetadata>.RemoveAt(int) , Collection<ParameterMetadata>.Count , Collection<ParameterMetadata>.this[int] , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Methods

Add(ParameterMetadata?)

Adds the specified parameter metadata.

```
public ParameterCollection Add(ParameterMetadata? metadata)
```

Parameters

`metadata` ParameterMetadata

The metadata to add.

Returns

ParameterCollection

The current ParameterCollection instance.

Exceptions

ArgumentNullException

`metadata` is null.

Add(IDbDataParameter?)

Adds a provider-specific parameter by copying its metadata.

```
public ParameterCollection Add(IDbDataParameter? parameter)
```

Parameters

`parameter` IDbDataParameter

The provider parameter to copy.

Returns

ParameterCollection

The current ParameterCollection instance.

Exceptions

ArgumentNullException

`parameter` is null.

Add(string, object?)

Adds a parameter by name and value.

```
public ParameterCollection Add(string parameterName, object? value)
```

Parameters

`parameterName` string

The logical parameter name.

`value` object

The parameter value.

Returns

ParameterCollection

The current ParameterCollection instance.

Add(string, object?, DbType)

Adds a parameter by name, value, and explicit database type.

```
public ParameterCollection Add(string parameterName, object? value, DbType dbType)
```

Parameters

`parameterName` string

The logical parameter name.

`value` object

The parameter value.

`dbType` DbType

The database type.

Returns

ParameterCollection

The current ParameterCollection instance.

Add(string, object?, DbType, byte, byte)

Adds a parameter by name, value, database type, precision, and scale.

```
public ParameterCollection Add(string parameterName, object? value, DbType dbType, byte
precision, byte scale)
```

Parameters

`parameterName` string

The logical parameter name.

`value` object

The parameter value.

`dbType` DbType

The database type.

`precision` byte

The numeric precision.

`scale` byte

The numeric scale.

Returns

ParameterCollection

The current ParameterCollection instance.

Add(string, object?, DbType, ParameterDirection)

Adds a parameter by name, value, database type, and direction.

```
public ParameterCollection Add(string parameterName, object? value, DbType dbType,
ParameterDirection direction)
```

Parameters

`parameterName` string

The logical parameter name.

`value` object

The parameter value.

`dbType` DbType

The database type.

`direction` ParameterDirection

The parameter direction.

Returns

ParameterCollection

The current ParameterCollection instance.

Add(string, object?, DbType, int)

Adds a parameter by name, value, database type, and size.

```
public ParameterCollection Add(string parameterName, object? value, DbType dbType, int size)
```

Parameters

`parameterName` string

The logical parameter name.

`value` object

The parameter value.

`dbType` DbType

The database type.

`size` int

The parameter size.

Returns

ParameterCollection

The current ParameterCollection instance.

AddRange(params ParameterMetadata[])

Adds a range of ParameterMetadata objects.

```
public ParameterCollection AddRange(params ParameterMetadata[] parameters)
```

Parameters

`parameters` ParameterMetadata[]

The parameters to add.

Returns

ParameterCollection

The current ParameterCollection instance.

AddRange(IDictionary<string, object>)

Adds parameters from a dictionary of name/value pairs.

```
public ParameterCollection AddRange(IDictionary<string, object> pairs)
```

Parameters

`pairs` IDictionary<string, object>

The name/value pairs to add.

Returns

ParameterCollection

The current ParameterCollection instance.

Exceptions

ArgumentNullException

`pairs` is null.

AddRange(IEnumerable<ParameterMetadata>)

Adds a range of ParameterMetadata objects.

```
public ParameterCollection AddRange(IEnumerable<ParameterMetadata> parameters)
```

Parameters

`parameters` IEnumerable<ParameterMetadata>

The parameters to add.

Returns

ParameterCollection

The current ParameterCollection instance.

Exceptions

ArgumentNullException

`parameters` is null.

AddRange(IEnumerable<IDbDataParameter>)

Adds a range of provider-specific parameters by copying their metadata.

```
public ParameterCollection AddRange(IEnumerable<IDbDataParameter> parameters)
```

Parameters

`parameters` IEnumerable<IDbDataParameter>

The parameters to add.

Returns

ParameterCollection

The current ParameterCollection instance.

Exceptions

ArgumentNullException

`parameters` is null.

AddRange(params IDbDataParameter[])

Adds a range of provider-specific parameters by copying their metadata.

```
public ParameterCollection AddRange(params IDbDataParameter[] parameters)
```

Parameters

`parameters` IDbDataParameter[]

The parameters to add.

Returns

ParameterCollection

The current ParameterCollection instance.

AddRange(object)

Adds parameters from supported container types such as dictionaries, parameter collections, or anonymous objects.

```
public ParameterCollection AddRange(object obj)
```

Parameters

`obj` object

The source object.

Returns

ParameterCollection

The current ParameterCollection instance.

GetCommandExecutor()

Returns the owning CommandExecutor instance so that fluent chaining can continue.

```
public CommandExecutor GetCommandExecutor()
```

Returns

CommandExecutor

The owning CommandExecutor.

GetKeyForItem(ParameterMetadata)

```
protected override string GetKeyForItem(ParameterMetadata item)
```

Parameters

`item` ParameterMetadata

Returns

string

Class ParameterMetadata

Namespace: CloudyWing.DatabaseFacade

Assembly: CloudyWing.DatabaseFacade.dll

Describes the values used to create and configure a provider-specific database parameter.

```
public sealed class ParameterMetadata : ICloneable
```

Inheritance

object ← ParameterMetadata

Implements

ICloneable

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

ParameterMetadata()

Initializes a new instance of the ParameterMetadata class.

```
public ParameterMetadata()
```

Properties

DbType

Gets or sets the database type to apply when one is explicitly required.

```
public DbType? DbType { get; set; }
```

Property Value

DbType?

Direction

Gets or sets the parameter direction.

```
public ParameterDirection Direction { get; set; }
```

Property Value

ParameterDirection

ParameterName

Gets or sets the logical parameter name without the provider prefix. Leading `@`, `:`, or `?` characters are trimmed automatically.

```
public string? ParameterName { get; set; }
```

Property Value

string

Precision

Gets or sets the numeric precision.

```
public byte? Precision { get; set; }
```

Property Value

byte?

Scale

Gets or sets the numeric scale.

```
public byte? Scale { get; set; }
```

Property Value

byte?

Size

Gets or sets the parameter size.

```
public int? Size { get; set; }
```

Property Value

int?

SourceColumn

Gets or sets the source column name used by data adapters.

```
public string? SourceColumn { get; set; }
```

Property Value

string

SourceVersion

Gets or sets the DataRowVersion used by data adapters.

```
public DataRowVersion? SourceVersion { get; set; }
```

Property Value

DataRowVersion?

Value

Gets or sets the value assigned to the provider parameter.

```
public object? Value { get; set; }
```

Property Value

object

Methods

ApplyParameter(IDbDataParameter)

Copies the configured metadata to the specified provider parameter. Only explicitly assigned values are applied.

```
public void ApplyParameter(IDbDataParameter to)
```

Parameters

`to` IDbDataParameter

The provider parameter to configure.

Clone()

Creates a copy of the current parameter metadata.

```
public object Clone()
```

Returns

object

A new ParameterMetadata instance that contains the same values.

See Also

ICloneable

Enum ResetItems

Namespace: CloudyWing.DatabaseFacade

Assembly: CloudyWing.DatabaseFacade.dll

Specifies which parts of a CommandExecutor should be reset after an operation completes.

```
[Flags]
public enum ResetItems
```

Fields

`All = CommandText | CommandTimeout | CommandType | Parameters`

Reset all supported command state values.

`CommandText = 1`

Reset CommandText.

`CommandTimeout = 2`

Reset CommandTimeout.

`CommandType = 4`

Reset CommandType.

`None = 0`

Do not reset any command state.

`Parameters = 8`

Clear Parameters.